UCC 152-2 - Introduction to Security in DevOps

DigiBank Dynamic Security Analysis

Runtime security validation of the DigiBank API.

1. Why DAST is needed

Static analysis examines source code and configuration before execution. Dynamic analysis complements it by interacting with the application while it is running. This matters because a system can compile, pass unit tests, and still expose an API route without authentication, return excessive technical detail, accept invalid input, or handle sessions incorrectly.

DAST checks what an external client can actually observe, not only what the source code appears to intend.

The sequence is: establish runtime behavior, identify security weaknesses, apply targeted remediation, and replay the same scenarios to demonstrate the result.

2. Application under test

The external client does not call each business service directly. The API Gateway is the security boundary and the single entry point for the business API. Behind it, the customer, account, transaction, compliance, and notification services run as separate Compose services.

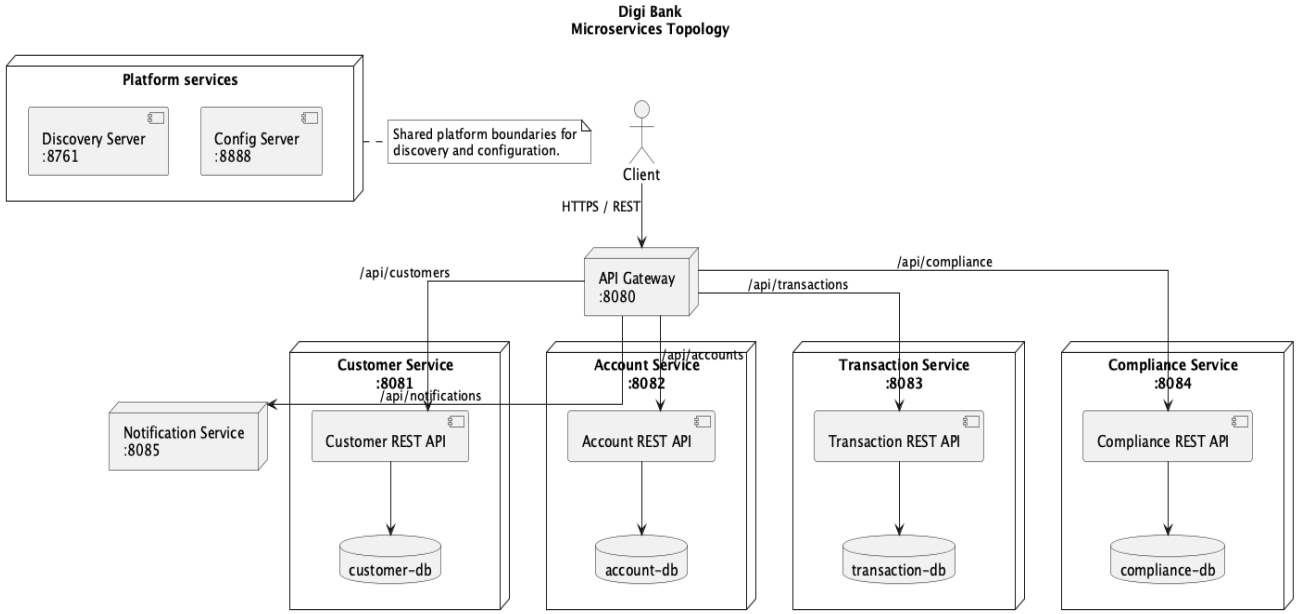


Figure 1. Runtime topology under test: gateway, business services, platform services, Compose host and persistence boundaries.

Surface	What is tested
Gateway	Authentication, authorization, headers, cache behavior, error responses and route protection.
Business APIs	Customer, account, transaction and compliance operations through gateway routing.
Runtime behavior	Malformed requests, invalid values, missing resources, session behavior and information disclosure.
Delivery path	Build, Compose startup, readiness, Newman replay and ZAP scanning.

3. Reproducible test environment

The environment is deliberately reproducible. Maven verifies the implementation, Docker Compose starts the complete topology, smoke checks verify the endpoints, and stack validation confirms that the expected containers are running before DAST begins.

```
make clean
make build
make image-build
make up
make smoke
make validate-stack
make newman-scan
make zap-scan
```

3.1 Starting the application

The Compose startup creates the complete local runtime. The screenshot shows the command used to start the services and the resulting container state. This establishes the execution context before any security request is sent.

```
➜  digibank-core git:(main) ✗ make up
/Applications/Xcode.app/Contents/Developer/usr/bin/make -C digibank-microservices up
docker compose -f docker-compose.yml up -d
[+] up 12/12
 ✔ Container digibank-microservices-config-server-1         Running                    0.0s
 ✔ Container digibank-microservices-transaction-db-1        Healthy                    1.2s
 ✔ Container digibank-microservices-compliance-db-1         Healthy                    0.7s
 ✔ Container digibank-microservices-compliance-service-1    Running                    0.0s
 ✔ Container digibank-microservices-account-db-1            Healthy                    0.7s
 ✔ Container digibank-microservices-account-service-1       Running                    0.0s
 ✔ Container digibank-microservices-notification-service-1  Running                    0.0s
 ✔ Container digibank-microservices-transaction-service-1   Running                    0.0s
 ✔ Container digibank-microservices-discovery-server-1      Running                    0.0s
 ✔ Container digibank-microservices-customer-db-1           Healthy                    0.7s
 ✔ Container digibank-microservices-customer-service-1      Running                    0.0s
 ✔ Container digibank-microservices-api-gateway-1           Running                    0.0s
```

The container view confirms that the gateway, business services, databases, discovery server and config server are present in the same runtime topology.

3.2 Smoke validation

Smoke validation checks the public health endpoint, gateway business routes, direct internal service endpoints, discovery registry and config server. It is not the security campaign; it proves that the application is alive and that later DAST failures can be interpreted as application behavior rather than startup failure.

```
→  digibank-core git:(main) ✗ make smoke
/Applications/Xcode.app/Contents/Developer/usr/bin/make -C digibank-microservices smoke
./scripts/smoke-test.sh
Checking gateway-health   http://localhost:8080/actuator/health
Checking customer-api     http://localhost:8080/api/customers
Checking account-api      http://localhost:8080/api/accounts
Checking transaction-api  http://localhost:8080/api/transactions
Checking compliance-api   http://localhost:8080/api/compliance
Checking customer-service http://localhost:8081/api/customers
Checking account-service  http://localhost:8082/api/accounts
Checking transaction-service http://localhost:8083/api/transactions
Checking compliance-service http://localhost:8084/api/compliance
Checking notification-service http://localhost:8085/actuator/health
Checking discovery-registry http://localhost:8761/registry
Checking config-server    http://localhost:8888/config/customer-service
Microservice smoke checks passed.
```

The successful result is the gate before Newman and ZAP.

4. Baseline and remediation decision

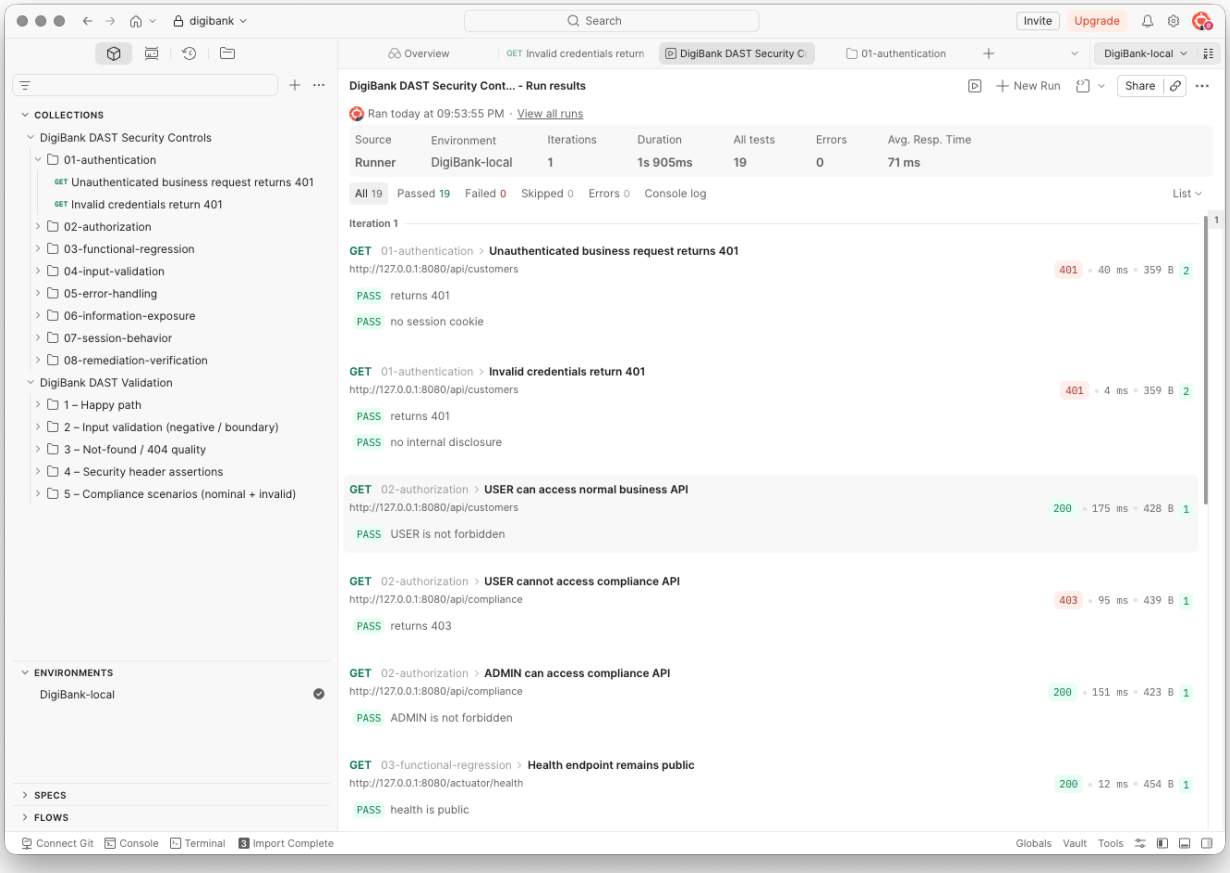
The baseline establishes the behavior before the gateway security boundary was introduced. The important observations were unauthenticated business routes, no gateway role distinction, incomplete response hardening, and no explicit cache policy for sensitive API responses.

The remediation uses stateless HTTP Basic authentication at the gateway because it is simple to reproduce in the workshop and preserves the existing service topology. It is not presented as the production identity architecture.

Request condition	Expected result after remediation
No credentials or invalid credentials	401 Unauthorized
Valid USER on normal banking API	Request is permitted
Valid USER on compliance API	403 Forbidden
Valid ADMIN on compliance API	Request is permitted
Authenticated request	No server session or session cookie

4.1 Authentication and authorization in Postman

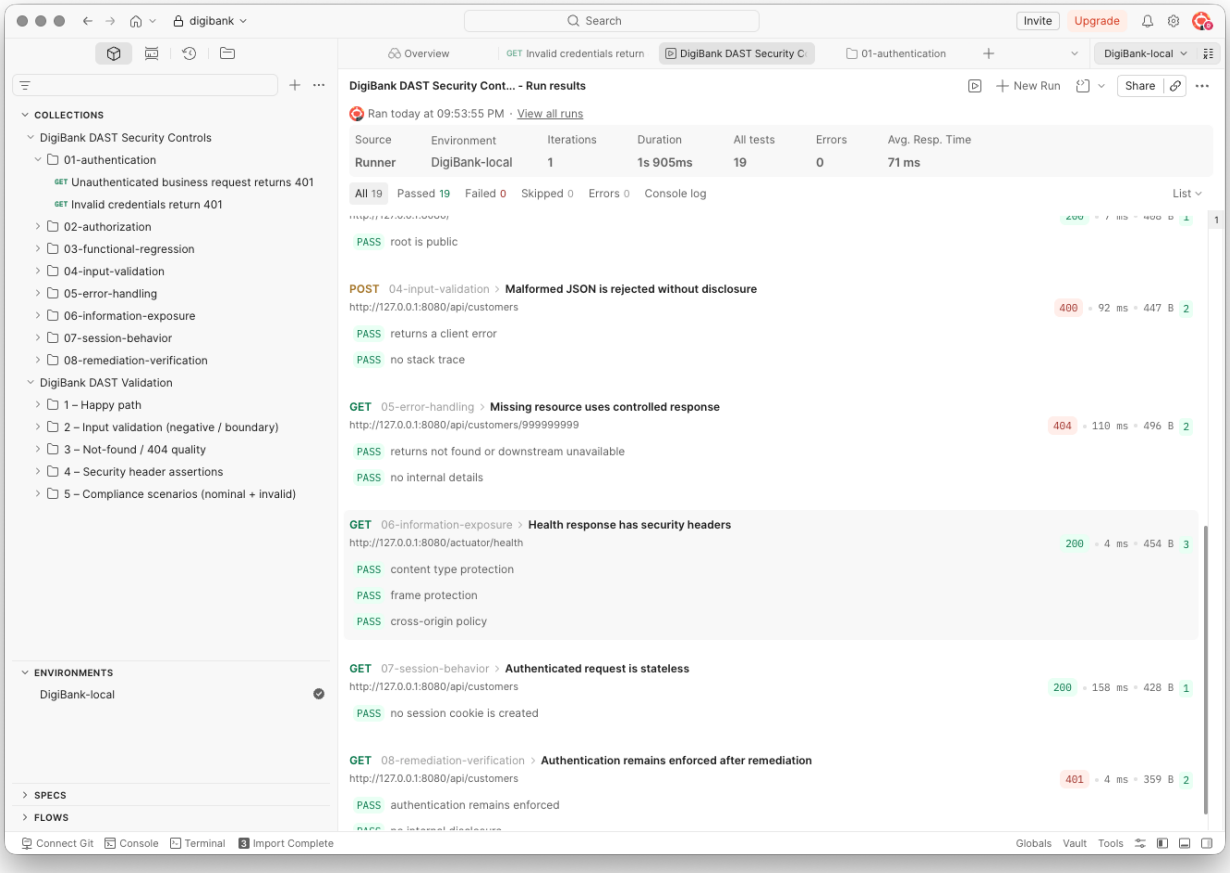
The security collection makes the access rules visible through actual requests. The run shows unauthenticated requests returning 401, normal USER access returning 200, USER access to compliance returning 403, and ADMIN access to compliance returning 200.



The request results correspond directly to the gateway rules: authentication establishes identity, and authorization decides whether that identity may use the route.

4.2 Validation, errors and information exposure

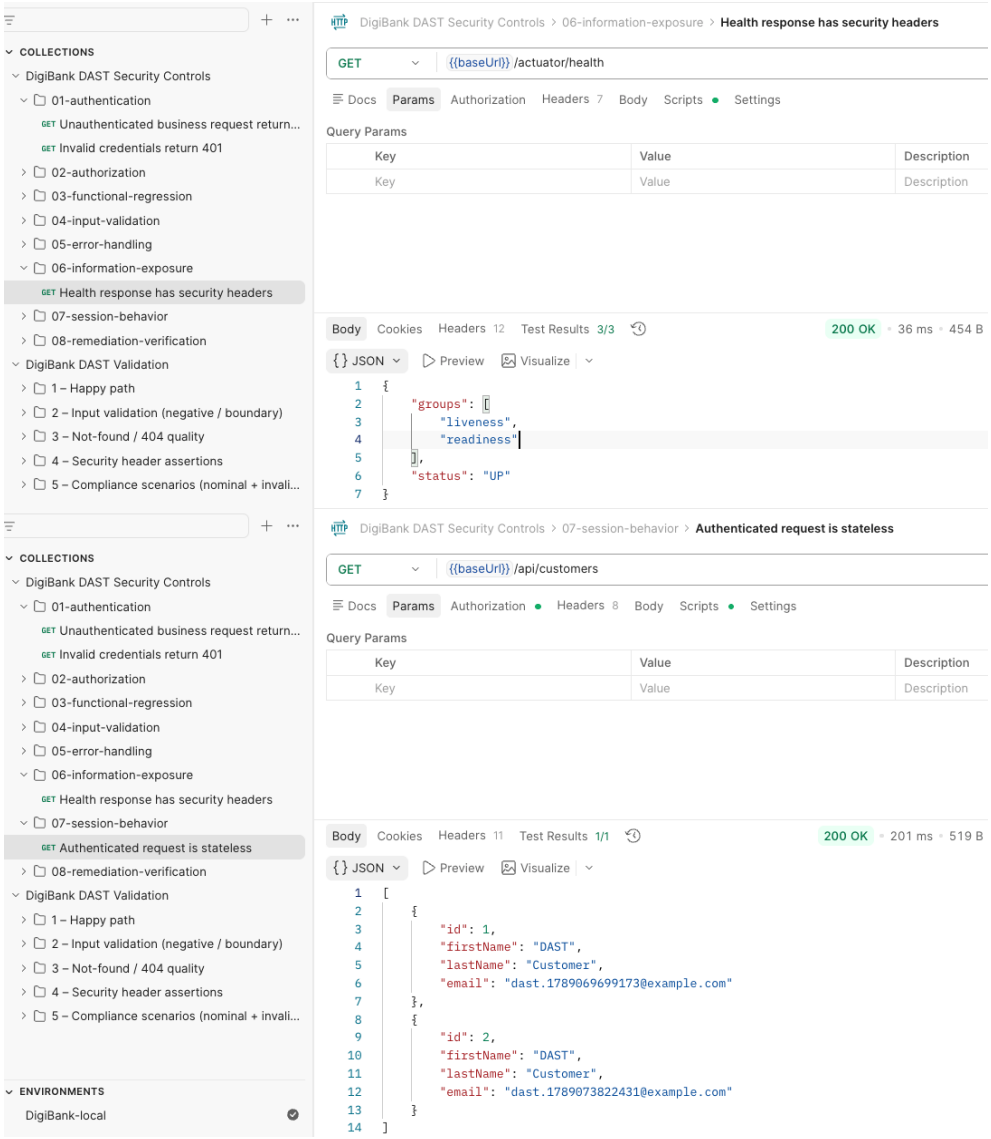
The security collection also checks malformed JSON and missing resources. These cases return controlled 4xx responses without stack traces or internal implementation details. The same collection verifies the response headers on the health endpoint.



The run shows the validation and error-handling assertions passing, including the controlled 400 and 404 responses.

4.3 Headers and stateless behavior

The headers check verifies browser and response protections. The session check verifies that an authenticated request does not create a server session or return a session cookie.



These screenshots show the two related controls: hardened responses and stateless authentication.

5. Runtime checks and scan results

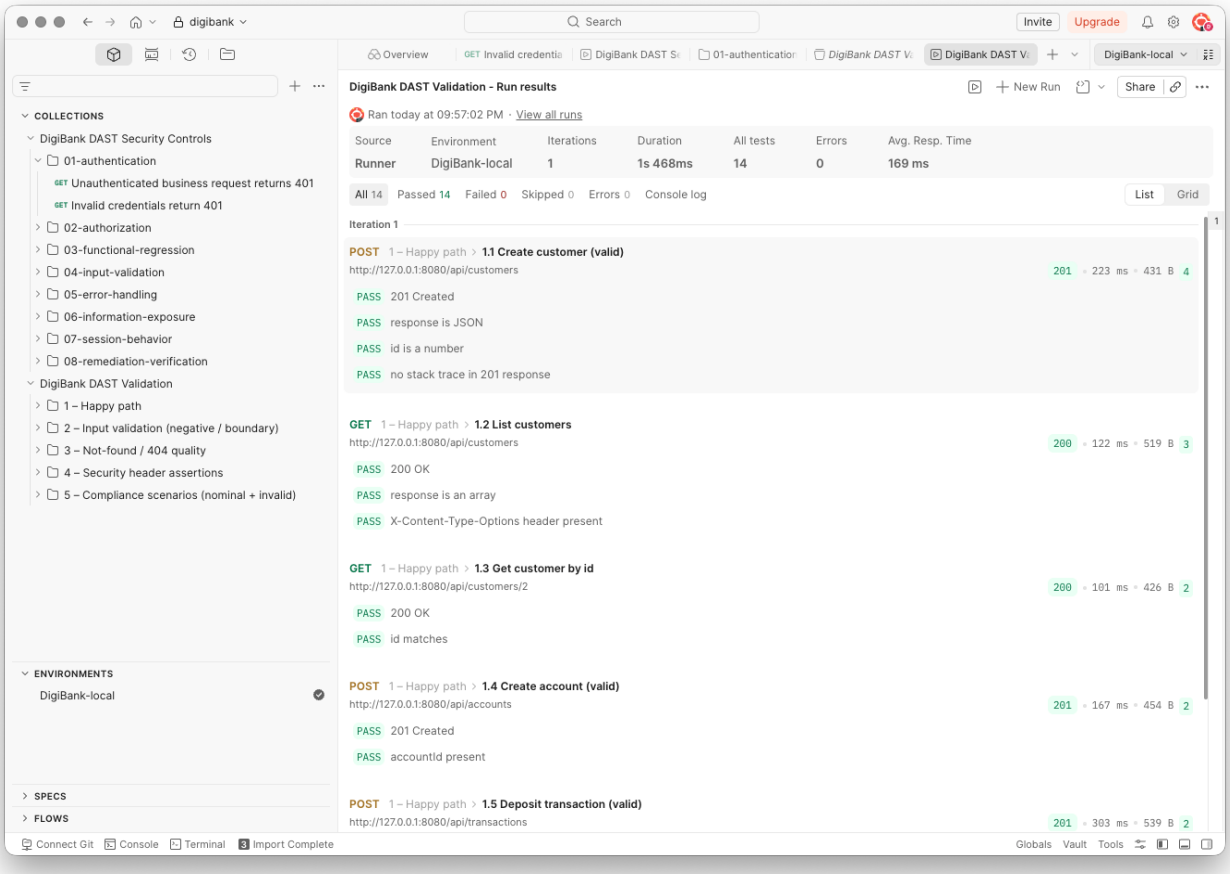
Newman expresses the expected security behavior as repeatable assertions. The security collection is divided into authentication, authorization, functional regression, input validation, error handling, information exposure, session behavior, and remediation verification.

Campaign	Result
Functional Newman collection	24 requests, 74 assertions, 0 failures.
Security Newman collection	12 requests, 19 assertions, 0 failures.
OWASP ZAP baseline	0 High, 1 Medium, 0 Low, 1 Informational.

The remaining Medium ZAP alert is the expected local warning that HTTP Basic credentials are sent over plain HTTP. It is acceptable only in this local educational environment and requires TLS in production. The Informational observation concerns public static-resource cacheability; authenticated API responses receive `Cache-Control: no-store, private`.

5.1 Functional regression in Postman

The functional collection checks the banking flows after the security changes: customer creation and retrieval, account creation, transactions and compliance operations. This confirms that the gateway protection did not break normal application behavior.



The collection run shows the banking requests and their passing assertions in the local environment.

5.2 Newman security replay

Newman repeats the same security collection without relying on the Postman interface. The summary is the reproducible command-line result used locally and in CI.

```
❑ 08-remediation-verification
→ Authentication remains enforced after remediation
  GET http://127.0.0.1:8080/api/customers [401 Unauthorized, 359B, 6ms]
  ✓  authentication remains enforced
  ✓  no internal disclosure

┌                         executed │           failed
├──────────────────────────────────┼──────────────────┤
│               iterations │     1 │                0 │
│                 requests │    12 │                0 │
│             test-scripts │    12 │                0 │
│       prerequest-scripts │     0 │                0 │
│               assertions │    19 │                0 │
└──────────────────────────────────┴──────────────────┘
  total run duration: 1074ms
  total data received: 1.24kB (approx)
  average response time: 64ms [min: 4ms, max: 200ms, s.d.: 59ms]

  [INFO]  Functional report: /Users/adorsys123/dev/digibank-core/dast/reports/202609
5/functional-newman-report.json
```

The security replay completed 12 requests and 19 assertions with zero failures.

The report directory contains separate functional and security JSON results.

5.3 OWASP ZAP scan

ZAP scans the externally visible gateway surface and complements the targeted Postman assertions. The scan checks the running application for observable response, header and authentication weaknesses that are not covered by a small set of targeted requests.

The scan completed with no High or Low alerts. The remaining findings are reviewed below and are interpreted against the local test environment rather than treated as an unexplained score.

5.4 ZAP alert summary

The ZAP summary reports zero High alerts, one Medium alert, zero Low alerts and one Informational alert. The remaining Medium alert is the local plain-HTTP warning associated with HTTP Basic authentication. It is recorded and explained in the report.

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	1
Low	0
Informational	1
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Info	Informational	http://127.0.0.1:8080	Percentage of responses with status code 2xx	50 %
Info	Informational	http://127.0.0.1:8080	Percentage of responses with status code 4xx	50 %
Info	Informational	http://127.0.0.1:8080	Percentage of endpoints with content type application/json	33 %
Info	Informational	http://127.0.0.1:8080	Percentage of endpoints with method GET	100 %
Info	Informational	http://127.0.0.1:8080	Count of total endpoints	3

6. CI validation

The CI workflow repeats the same validation chain after the application is built. Each step has a clear purpose: start the runtime, prove readiness, verify the exposed routes, replay the security assertions, and scan the gateway surface.

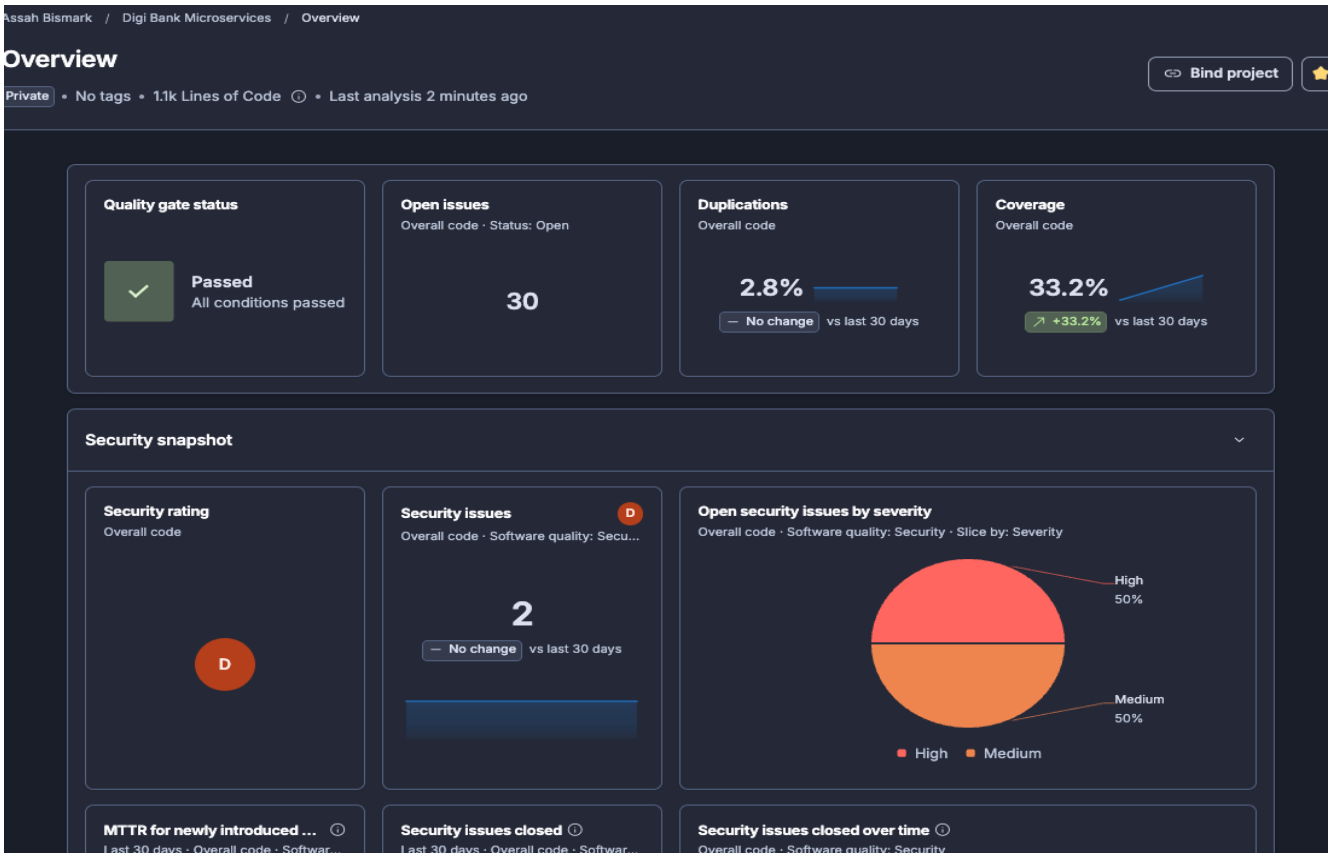
Pipeline step	Purpose
Build and tests	Compile the services and run the automated test suite.
Compose and readiness	Start the complete runtime and wait for the required endpoints.
Smoke validation	Confirm gateway, business APIs and platform services respond.
Newman	Run the versioned functional and security request assertions.
ZAP	Run an independent baseline scan against the running gateway.
Report upload	Keep Newman and ZAP results available when a check fails.

Newman and ZAP remain separate checks: Newman verifies expected security behavior, while ZAP performs broader automated observations of the exposed HTTP surface.

7. Static security analysis

Workshop 2 moves security checks earlier in the delivery lifecycle. Instead of waiting until the application is running, the source code, dependencies and test strength are inspected during the build. SonarCloud provides code and security analysis, JaCoCo supplies test coverage data, and PITest measures whether tests detect deliberately introduced defects.

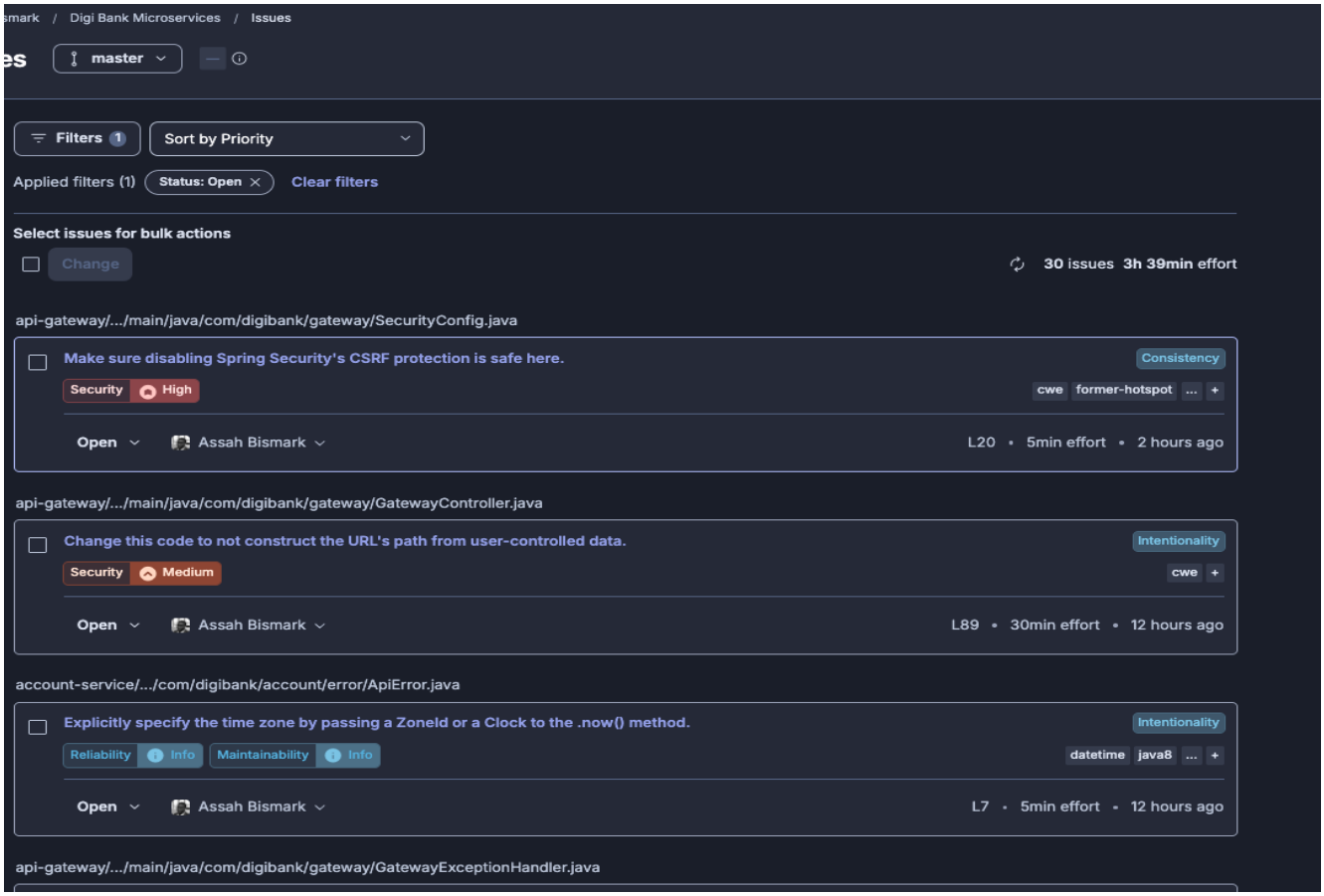
The analysis is applied to the current Java 21 Spring Boot microservices implementation. It does not replace the Newman and ZAP runtime checks; it complements them.



SonarCloud quality gate passed. The dashboard reports 33.2% overall coverage, 2.8% duplication and 30 open issues.

7.1 SonarCloud findings and decisions

The open-issues view identified two security issues. The CSRF warning is an intentional decision for this workshop because the gateway uses stateless HTTP Basic authentication and does not create browser sessions or cookies. The URL-construction warning was treated as a real issue and remediated by encoding gateway path variables before forwarding them to downstream services.

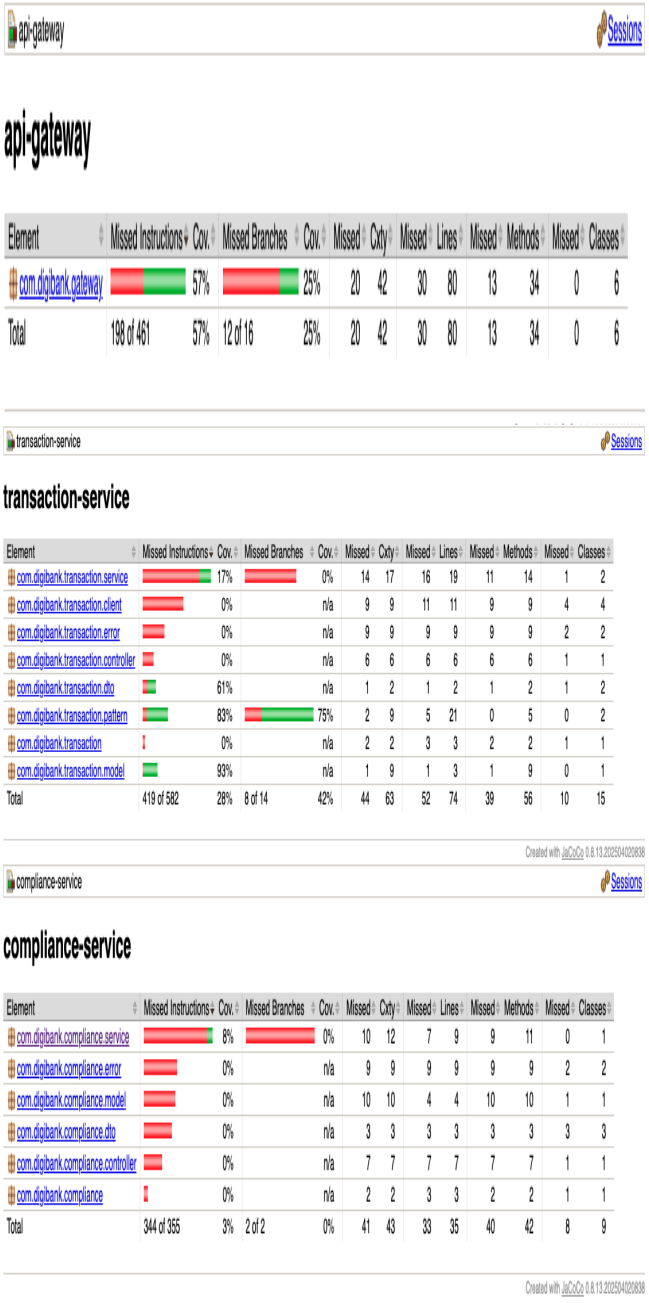


Informational timestamp findings using LocalDateTime.now() are recorded as a maintainability follow-up. They do not represent an authorization, injection or secret-exposure defect.

7.2 JaCoCo coverage evidence

JaCoCo records which production lines are exercised by the automated tests. The report is generated during Maven verify and is imported by SonarCloud. Coverage is evidence about test reach; it is not the same as proving that the tests detect incorrect behavior.

```
→  digibank-core git:(main) × find digibank-microservices -path '*/target/site/jaco
.xml'
digibank-microservices/compliance-service/target/site/jacoco/jacoco.xml
digibank-microservices/account-service/target/site/jacoco/jacoco.xml
digibank-microservices/api-gateway/target/site/jacoco/jacoco.xml
digibank-microservices/customer-service/target/site/jacoco/jacoco.xml
digibank-microservices/transaction-service/target/site/jacoco/jacoco.xml
digibank-microservices/config-server/target/site/jacoco/jacoco.xml
digibank-microservices/discovery-server/target/site/jacoco/jacoco.xml
```



api-gateway Sessions

api-gateway

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.digibank.gateway		57%		25%	20	42	30	80	13	34	0	6
Total	198 of 461	57%	12 of 16	25%	20	42	30	80	13	34	0	6

transaction-service Sessions

transaction-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.digibank.transaction.service		17%		0%	14	17	16	19	11	14	1	2
com.digibank.transaction.client		0%		n/a	9	9	11	11	9	9	4	4
com.digibank.transaction.error		0%		n/a	9	9	9	9	9	9	2	2
com.digibank.transaction.controller		0%		n/a	6	6	6	6	6	6	1	1
com.digibank.transaction.dto		61%		n/a	1	2	1	2	1	2	1	2
com.digibank.transaction.pattern		83%		75%	2	9	5	21	0	5	0	2
com.digibank.transaction		0%		n/a	2	2	3	3	2	2	1	1
com.digibank.transaction.model		93%		n/a	1	9	1	3	1	9	0	1
Total	419 of 582	28%	8 of 14	42%	44	63	52	74	39	56	10	15

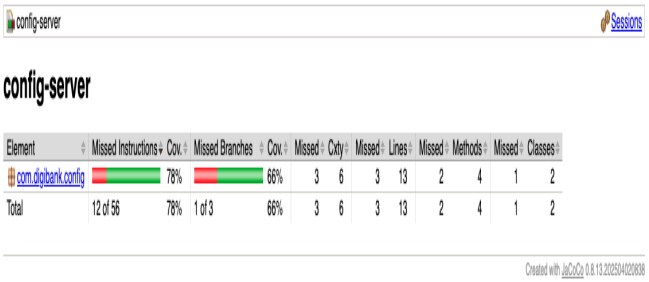
Created with JaCoCo 0.8.13.202504020838

compliance-service Sessions

compliance-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.digibank.compliance.service		8%		0%	10	12	7	9	9	11	0	1
com.digibank.compliance.error		0%		n/a	9	9	9	9	9	9	2	2
com.digibank.compliance.model		0%		n/a	10	10	4	4	10	10	1	1
com.digibank.compliance.dto		0%		n/a	3	3	3	3	3	3	3	3
com.digibank.compliance.controller		0%		n/a	7	7	7	7	7	7	1	1
com.digibank.compliance		0%		n/a	2	2	3	3	2	2	1	1
Total	344 of 355	3%	2 of 2	0%	41	43	33	35	40	42	8	9

Created with JaCoCo 0.8.13.202504020838


config-server

Sessions

config-server

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.digibank.config		78%		66%	3	6	3	13	2	4	1	2
Total	12 of 56	78%	1 of 3	66%	3	6	3	13	2	4	1	2

Created with JaCoCo 0.8.13.202504020838

Coverage reports are available for the tested services. The notification service has no test sources and therefore produces no execution data.

7.3 PITest mutation testing

PITest changes compiled bytecode in controlled ways, such as negating a condition or replacing a return value. A killed mutant means a test detected the change. A surviving or uncovered mutant identifies a branch where the current tests are not strong enough to detect that behavior change.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
6	63% 50/80	12% 4/34	24% 4/17

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.digibank.gateway	6	63% 50/80	12% 4/34	24% 4/17

- Project uses Spring, but the Arcmutate Spring plugin is not present.

Report generated by PIT 1.17.4

Enhanced functionality available at arcmutate.com

The gateway report generated 34 mutations, killed 4, and recorded 24% test strength. This is a test-strength observation, not a failed scan.

Pit Test Coverage Report

Project Summary

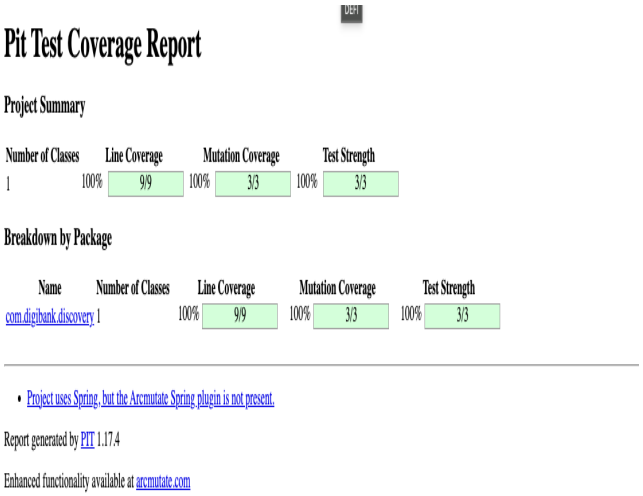
Number of Classes	Line Coverage	Mutation Coverage	Test Strength
5	27% 12/44	10% 3/29	50% 3/6

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.digibank.customer.controller	1	0% 0/6	0% 0/5	100% 0/0
com.digibank.customer.error	2	0% 0/9	0% 0/7	100% 0/0
com.digibank.customer.model	1	64% 9/14	25% 1/4	25% 1/4
com.digibank.customer.service	1	20% 3/15	15% 2/13	100% 2/2

- Project uses Spring, but the Arcmutate Spring plugin is not present.

Report generated by PIT 1.17.4



Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
1	100%	9/9	100%	3/3	100%	3/3

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
com.digibank.discovery	1	100%	9/9	100%	3/3	100%	3/3

- Project uses Spring, but the Arcmutate Spring plugin is not present.

Report generated by PIT 1.17.4

Enhanced functionality available at arcmutate.com

The other service reports show that mutation results are module-specific; they should be reviewed as test-improvement input rather than combined into one misleading score.

7.4 Static-analysis delivery controls

Static checks are exposed through simple Make commands so the instructor and the team can reproduce the same controls locally.

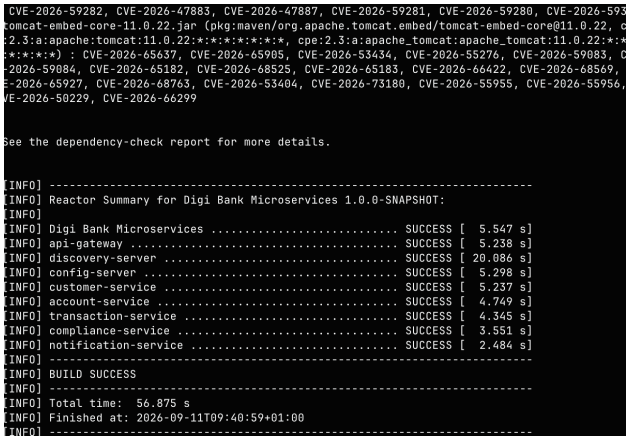
```
make build
make sast-sonar
make sast-pitest
make sast-dependency-check
```

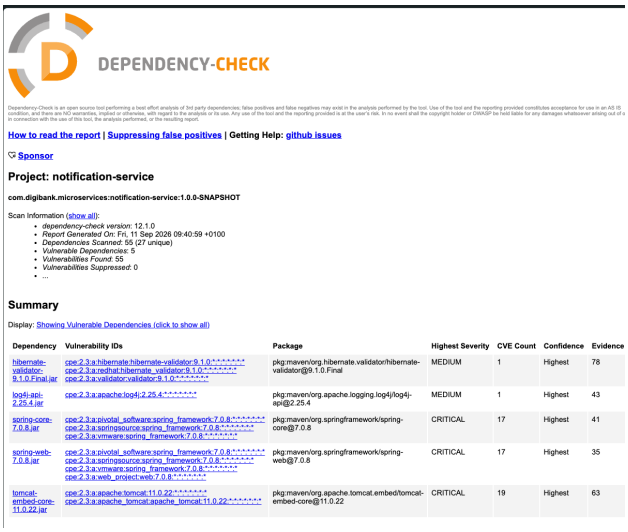
The CI workflow runs SonarCloud as a separate job, runs Dependency-Check and PITest in the static-analysis job, uploads reports even when a scan fails, and caches the Dependency-Check NVD database. The first NVD database download is large; later runs reuse the GitHub Actions cache and retrieve only updates.

Control	Evidence and decision
SonarCloud	Quality gate passed; issues reviewed and classified.
JaCoCo	Coverage generated and imported into SonarCloud.
PITest	Mutation reports generated; surviving mutants become test-strength follow-up.
Dependency-Check	NVD report generated; zero unsuppressed CVEs after the Tomcat patch and reviewed false-positive rules.
CI	Separate jobs, secret-based credentials, report upload and cache reuse.

7.5 Dependency-Check results

The NVD update completed and the dependency scan generated HTML and JSON reports. The initial API Gateway report contained 43 unique CVE identifiers. After upgrading Spring Boot to 4.1.1, overriding embedded Tomcat to 11.0.25 and rerunning the scan, no unsuppressed CVEs remain across the nine Maven projects. Two dependency matches were confirmed as false positives and are covered by narrowly scoped suppression rules.





DEPENDENCY-CHECK

How to read the report | Suppressing false positives | Getting Help: github issues

Sponsor

Project: notification-service

com.digibank.microservices:notification-service:1.0.0-SNAPSHOT

Scan Information (show all):

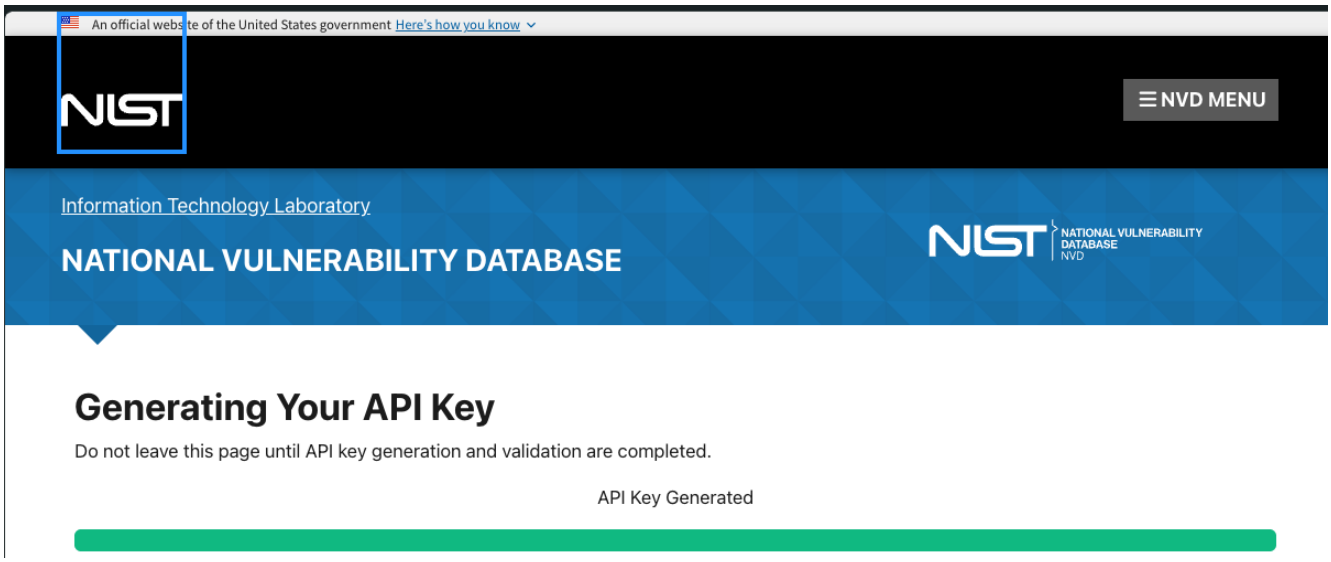
- *dependency-check version:* 12.1.0
- *Report Generated On:* Fri, 11 Sep 2026 09:40:59 +0100
- *Dependencies Scanned:* 55 (27 unique)
- *Vulnerable Dependencies:* 5
- *Vulnerabilities Found:* 55
- *Vulnerabilities Suppressed:* 0
- ...

Summary

Display: Showing Vulnerable Dependencies (click to show all)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence
hibernate-validator-9.1.0.Final.jar	cpe:2.3:a:hibernate:hibernate-validator:9.1.0:*:*:*:*:*:*:* cpe:2.3:a:redhat:hibernate_validator:9.1.0:*:*:*:*:*:*:* cpe:2.3:a:validator:validator:9.1.0:*:*:*:*:*:*:*	pkg:maven/org.hibernate.validator/hibernate-validator@9.1.0.Final	MEDIUM	1	Highest	78
log4j-api-2.25.4.jar	cpe:2.3:a:apache:log4j:2.25.4:*:*:*:*:*:*:*	pkg:maven/org.apache.logging.log4j/log4j-api@2.25.4	MEDIUM	1	Highest	43
spring-core-7.0.8.jar	cpe:2.3:a:pivotal_software:spring_framework:7.0.8:*:*:*:*:*:*:* cpe:2.3:a:springsource:spring_framework:7.0.8:*:*:*:*:*:*:* cpe:2.3:a:vmware:spring_framework:7.0.8:*:*:*:*:*:*:*	pkg:maven/org.springframework/spring-core@7.0.8	CRITICAL	17	Highest	41
spring-web-7.0.8.jar	cpe:2.3:a:pivotal_software:spring_framework:7.0.8:*:*:*:*:*:*:* cpe:2.3:a:springsource:spring_framework:7.0.8:*:*:*:*:*:*:* cpe:2.3:a:vmware:spring_framework:7.0.8:*:*:*:*:*:*:* cpe:2.3:a:web_project:web:7.0.8:*:*:*:*:*:*:*	pkg:maven/org.springframework/spring-web@7.0.8	CRITICAL	17	Highest	35
tomcat-embed-core-11.0.22.jar	cpe:2.3:a:apache:tomcat:11.0.22:*:*:*:*:*:*:* cpe:2.3:a:apache_tomcat:apache_tomcat:11.0.22:*:*:*:*:*:*:*	pkg:maven/org.apache.tomcat.embed/tomcat-embed-core@11.0.22	CRITICAL	19	Highest	63

The captured NVD run shows the completed update, while the Dependency-Check report shows the scan output. The NIST source reference is retained as supporting context for the vulnerability database used by the scan.



An official website of the United States government Here's how you know

NIST

NVD MENU

Information Technology Laboratory

NATIONAL VULNERABILITY DATABASE

NIST NATIONAL VULNERABILITY DATABASE NVD

Generating Your API Key

Do not leave this page until API key generation and validation are completed.

API Key Generated

The false-positive decisions are explicit: Angus Activation was matched to a Jakarta Mail SMTP issue, and Hibernate Validator was matched to the unrelated Nu Html Checker issue. The suppression file is versioned and limited to those CVE/component combinations.

Result	Decision
NVD data	Successfully updated and reused from the local/CI cache.
Report formats	HTML and JSON generated for the parent and service modules.
OSS Index	Disabled because the unauthenticated endpoint returned HTTP 401.
Before remediation	43 unique API Gateway CVEs: 15 Critical, 13 High, 14 Medium, 1 Low.
After remediation	Zero unsuppressed CVEs after Tomcat 11.0.25 and reviewed false-positive rules.
Evidence	dependency-check-report.html, dependency-check-report.json and dependency-check-suppressions.xml.

7.6 Workshop 2 completion status

The static-analysis setup is operational and reproducible. SonarCloud with JaCoCo, PITest, Dependency-Check, Maven verification and CI report-upload controls have all been executed. The

justified code remediation identified by SonarCloud was applied, and the dependency baseline now has zero unsuppressed CVEs after the Tomcat patch and reviewed false-positive decisions.

Workshop 2 is complete as a documented analysis and remediation exercise. PITest surviving mutants remain test-strength follow-up items, not unresolved dependency vulnerabilities; the reports preserve them for future test improvement.

8. Deliverable mapping

Objective	Demonstrated by
A2.1-A2.2	SAST/Shift Left explanation and source-security review scope.
A2.3	Review of validation, information exposure, dependency and test-strength risks.
A2.4	Maven, SonarCloud, JaCoCo, PITest, Dependency-Check and GitHub Actions.
A2.5	SonarCloud findings interpreted as remediation, accepted decision or follow-up.
A2.6-A2.7	Gateway path hardening, PITest configuration fix and documented security decisions.
A2.8	Passing build/tests, runtime DAST evidence and reproducible static-analysis controls.
A3.1-A3.8	Workshop 1 Newman/ZAP report and runtime evidence retained in this continuation PDF.

The result is a documented chain from source and dependency analysis, to risk interpretation, to remediation, to runtime revalidation.